Introduction to Parallel Algorithms and Programming

TD2: Communication and algorithms on a ring

Oguz Kaya

Assistant Professor
Université Paris-Saclay and LISN, Gif-sur-Yvette, France

université PARIS-SACLAY

Conventions

- The exercises we cannot cover in class are homework: solutions will be posted. The algorithms of an exercise may be used in the following ones.
- The **unidirectional ring** of the lecture: p processors $P_0, \ldots, P_{p-1}$ with their own memory; P_k has one link out, to P_{k+1}, and one link in, from P_{k-1} (indices modulo p).
- Four routines: MyRank(), NumProcs(), Send(msg, m) to P_{k+1}, Recv(msg, m) from P_{k-1}; arrays are passed by name, their size as a separate parameter; $msg[a \ldots]$ is the array starting at its element a, $M[j]$ the row j of a 2-D array; Send does not block, Recv blocks, Send(...) $\|$ Recv(...) runs both at once.
- A message of m elements costs $L + m\,b$ on a link; different links work at the same time, the same link one message at a time; an operation costs τ, with $\tau \leq b \ll L$ ($b \approx 10$ to $100\,\tau$: nanoseconds; L: microseconds). Cost: the time until the last processor is done, split into T_{comm} and T_{comp}.
- Building blocks from the lecture, and their pipelined versions of Exercises 2 and 3 (same signatures) once established; their costs are the object of Exercises 2 and 3:

Algorithm	Effect
Bcast(k, msg, m)	the array msg of P_k is copied into the msg of every other processor
Reduce($k, msg, m, \otimes$)	P_k gets in msg the element-wise $\otimes$ of the arrays msg of all; the others keep theirs

Outline

1. Exercise 1: Pipelined point-to-point communication
2. Exercise 2: Pipelined broadcast on a ring
3. Exercise 3: Reduction on a ring
4. Exercise 4: Scan on a ring
5. Exercise 5: Matrix–vector product on a ring
6. Exercise 6: Scatter on a ring
7. Exercise 7: Transposition on a ring
8. Exercise 8: Grids, hypercubes and binary trees

Exercise 1: Pipelined point-to-point communication

Processor P_i must send a message of m elements, stored in its array *msg*, to processor P_j ($j \neq i$); on the ring, the message has to go through $P_{i+1}, \ldots, P_{j-1}$ (indices modulo p: the message may wrap around the ring) and crosses $d = (j - i) \bmod p$ links. In the *store-and-forward* protocol, every intermediate processor receives the whole message, then forwards it, which costs $c_{i,j}(m) = d\,(L + m\,b)$: a single link is busy at a time, while the $d - 1$ others are idle. To use them, we *pipeline* the transfer: the message is cut into r pieces of $m' = m/r$ elements (assume that r divides m), sent one after the other; every intermediate processor forwards the piece it received at the previous step while it receives the next one.

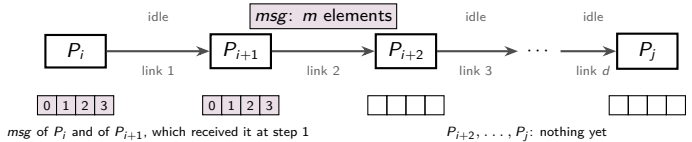
a) Write the algorithm SendTo(i, j, msg, m, r), executed by every processor, that performs this pipelined transfer; the processors outside $P_i, \ldots, P_j$ do nothing, and P_j receives the message in its own array *msg*.

b) Find the cost $c_{i,j}(m, r)$ of the pipelined transfer, in terms of r, d, L, m and b.

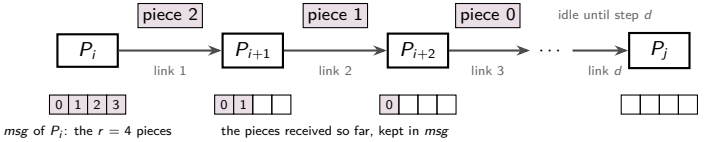
c) Which value of r minimizes this cost? What is the cost then, and what does it become for large messages? Compare with the store-and-forward cost.

Exercise 1: store-and-forward versus pipelined transfer

store-and-forward, at step 2: the whole message crosses link 2; the other links are idle



pipelined, $r = 4$ pieces of $m' = m/r$ elements, at step 3: pieces 2, 1, 0 cross links 1, 2, 3 at the same time



- Store-and-forward: the whole message hops from link to link, d steps of $L + m\,b$, **one link busy at a time**.
- Pipelined: r pieces of $m' = m/r$ elements follow each other one step apart; from step d on, **all the links are busy**.

Exercise 1: Pipelined point-to-point communication

Processor P_i must send a message of m elements, stored in its array *msg*, to processor P_j ($j \neq i$); on the ring, the message has to go through $P_{i+1}, \ldots, P_{j-1}$ (indices modulo p: the message may wrap around the ring) and crosses $d = (j - i) \bmod p$ links. In the *store-and-forward* protocol, every intermediate processor receives the whole message, then forwards it, which costs $c_{i,j}(m) = d\,(L + m\,b)$: a single link is busy at a time, while the $d - 1$ others are idle. To use them, we *pipeline* the transfer: the message is cut into r pieces of $m' = m/r$ elements (assume that r divides m), sent one after the other; every intermediate processor forwards the piece it received at the previous step while it receives the next one.

a) Write the algorithm SendTo(i, j, msg, m, r), executed by every processor, that performs this pipelined transfer; the processors outside $P_i, \ldots, P_j$ do nothing, and P_j receives the message in its own array *msg*.

b) Find the cost $c_{i,j}(m, r)$ of the pipelined transfer, in terms of r, d, L, m and b.

c) Which value of r minimizes this cost? What is the cost then, and what does it become for large messages? Compare with the store-and-forward cost.

Exercise 1(a): SendTo, the pipelined transfer

- Piece t is the sub-array $msg[t\,s \ldots (t+1)s-1]$, $s = m/r$.
- P_i sends the r pieces one after the other (line 3); P_j receives them one after the other (line 5).
- An intermediate processor (line 6: strictly between P_i and P_j along the ring) receives piece 0 (line 7), then **forwards piece $t-1$ while it receives piece t** (line 9: both links at once), and forwards the last piece (line 10).
- Every piece lands at its place in the processor's own *msg*: the intermediate processors get a copy too; pieces $t-1$ and t occupy different cells.

SendTo(i, j, msg, m, r)

```
1:  q ← MyRank(); p ← NumProcs(); s ← m/r
2:  if q = i then
3:    for t = 0, ..., r − 1 do Send(msg[t s ...], s)
4:  else if q = j then
5:    for t = 0, ..., r − 1 do Recv(msg[t s ...], s)
6:  else if 0 < (q − i) mod p < (j − i) mod p then
7:    Recv(msg[0 ...], s)
8:    for t = 1, ..., r − 1 do
9:      Send(msg[(t − 1)s ...], s) ∥ Recv(msg[t s ...], s)
10:   Send(msg[(r − 1)s ...], s)
```

Exercise 1: Pipelined point-to-point communication

Processor P_i must send a message of m elements, stored in its array *msg*, to processor P_j ($j \neq i$); on the ring, the message has to go through $P_{i+1}, \ldots, P_{j-1}$ (indices modulo p: the message may wrap around the ring) and crosses $d = (j - i) \bmod p$ links. In the *store-and-forward* protocol, every intermediate processor receives the whole message, then forwards it, which costs $c_{i,j}(m) = d\,(L + m\,b)$: a single link is busy at a time, while the $d - 1$ others are idle. To use them, we *pipeline* the transfer: the message is cut into r pieces of $m' = m/r$ elements (assume that r divides m), sent one after the other; every intermediate processor forwards the piece it received at the previous step while it receives the next one.

a) Write the algorithm SendTo(i, j, msg, m, r), executed by every processor, that performs this pipelined transfer; the processors outside $P_i, \ldots, P_j$ do nothing, and P_j receives the message in its own array *msg*.

b) Find the cost $c_{i,j}(m, r)$ of the pipelined transfer, in terms of r, d, L, m and b.

c) Which value of r minimizes this cost? What is the cost then, and what does it become for large messages? Compare with the store-and-forward cost.

Exercise 1(b): which piece crosses which link, and when

Link l goes from P_{i+l-1} to P_{i+l}. Piece 0 crosses link 1 at step 1, link 2 at step 2, ...; piece t follows one step behind on every link: it crosses link l at **step $l+t$**. The last piece crosses the last link at step $d+r-1$, every step costs $L+(m/r)\,b$: $c_{i,j}(m,r) = (d+r-1)\left(L+\frac{m}{r}\,b\right)$, which gives $d\,(L+m\,b)$ for $r=1$.

step	1	2	3
$P_i \to P_{i+1}$	the whole message		
$P_{i+1} \to P_{i+2}$		the whole message	
$P_{i+2} \to P_j$			the whole message

step	1	2	3	4	5	6
$P_i \to P_{i+1}$	0	1	2	3		
$P_{i+1} \to P_{i+2}$		0	1	2	3	
$P_{i+2} \to P_j$			0	1	2	3

$d=3$ links; store-and-forward on the left, $r=4$ pieces on the right; the width of a cell is the duration of the step ($L+m\,b$ and $L+(m/4)\,b$, here with $L=m\,b/4$).

Exercise 1: Pipelined point-to-point communication

Processor P_i must send a message of m elements, stored in its array *msg*, to processor P_j ($j \neq i$); on the ring, the message has to go through $P_{i+1}, \ldots, P_{j-1}$ (indices modulo p: the message may wrap around the ring) and crosses $d = (j - i) \bmod p$ links. In the *store-and-forward* protocol, every intermediate processor receives the whole message, then forwards it, which costs $c_{i,j}(m) = d\,(L + m\,b)$: a single link is busy at a time, while the $d - 1$ others are idle. To use them, we *pipeline* the transfer: the message is cut into r pieces of $m' = m/r$ elements (assume that r divides m), sent one after the other; every intermediate processor forwards the piece it received at the previous step while it receives the next one.

a) Write the algorithm SendTo(i, j, msg, m, r), executed by every processor, that performs this pipelined transfer; the processors outside $P_i, \ldots, P_j$ do nothing, and P_j receives the message in its own array *msg*.

b) Find the cost $c_{i,j}(m, r)$ of the pipelined transfer, in terms of r, d, L, m and b.

c) Which value of r minimizes this cost? What is the cost then, and what does it become for large messages? Compare with the store-and-forward cost.

Exercise 1(c): the best number of pieces

- $f(r) = c_{i,j}(m, r) = (d-1)\,L + m\,b + r\,L + (d-1)\,m\,b/r$: the two first terms do not depend on r.
- Derivative: $f'(r) = L - (d-1)\,m\,b/r^2$; $f'(r) = 0 \iff r^2 = (d-1)\,m\,b/L \iff r^\star = \sqrt{(d-1)\,m\,b/L}$ (rounded to an integer dividing m). $f'' > 0$: a **minimum**, not a maximum (see the solutions).
- At $r^\star$, $r\,L = (d-1)\,m\,b/r = \sqrt{(d-1)\,L\,m\,b}$, so $c_{i,j}(m, r^\star) = (d-1)\,L + m\,b + 2\sqrt{(d-1)\,L\,m\,b}$.
- Large messages: $c_{i,j} \approx m\,b$, the cost over a **single link**, instead of $d\,m\,b$: the pipeline **hides the distance**; what remains of it, $(d-1)\,L + 2\sqrt{(d-1)\,L\,m\,b}$, grows like $\sqrt{m}$ only.
- Small messages ($m\,b \ll L$): $r^\star < 1$, so $r = 1$, store-and-forward: the d latencies are paid anyway.

Algorithm	T_{comm}	T_{comp}	Remark
store-and-forward	$d\,(L + m\,b)$	0	one link busy at a time
SendTo, r pieces	$(d + r - 1)(L + m\,b/r)$	0	all the links busy after d steps
SendTo, $r = r^\star$	$(d-1)\,L + m\,b + 2\sqrt{(d-1)\,L\,m\,b}$	0	$\approx m\,b$ for large m

Exercise 1: the same idea in hardware, cut-through routing

- The pipelined transfer is the idea behind the **cut-through** (or **wormhole**) routing of network hardware: a switch forwards the units of a message (the **flits**, a few bytes) as soon as they arrive, instead of storing the whole message first; a message in transit is spread over all the links of its path.
- The difference: in hardware, a flit pays only a small per-hop delay $t_h \ll L$, not the latency L of a message: cost $L + d\,t_h + m\,b$, nothing to choose; here every piece is a message of its own and pays L on every link, hence the compromise on r.
- In both cases the distance disappears from the transmission term: $m\,b$ once, instead of d times.

Algorithm	T_{comm}	T_{comp}	Remark
store-and-forward	$d\,(L + m\,b)$	0	the whole message at each hop
SendTo, $r = r^\star$ (software)	$(d-1)\,L + m\,b + 2\sqrt{(d-1)\,L\,m\,b}$	0	every piece pays L at each hop
cut-through (hardware)	$L + d\,t_h + m\,b$	0	a flit pays $t_h \ll L$ at each hop

Outline

1. Exercise 1: Pipelined point-to-point communication
2. Exercise 2: Pipelined broadcast on a ring
3. Exercise 3: Reduction on a ring
4. Exercise 4: Scan on a ring
5. Exercise 5: Matrix–vector product on a ring
6. Exercise 6: Scatter on a ring
7. Exercise 7: Transposition on a ring
8. Exercise 8: Grids, hypercubes and binary trees

Exercise 2: Pipelined broadcast on a ring

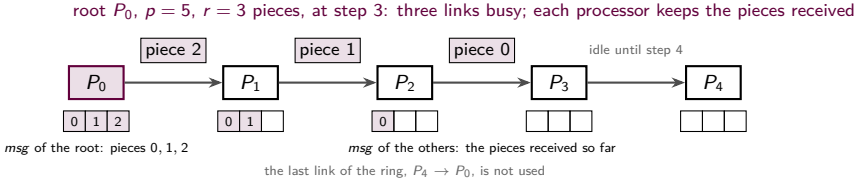
The $\textsc{Bcast}(k, msg, m)$ of the lecture relays the whole array of the root around the ring, in $(p-1)(L+m\,b)$: again a single link is busy at a time. Pipelining applies: the root cuts its array into r pieces of m/r elements (r divides m) and sends them one after the other; every other processor forwards each piece as soon as it has it, while it receives the next one, and keeps a copy. The number of pieces r is not a parameter: the algorithm chooses it internally, as well as possible (question (c)).

a) Write the pipelined algorithm $\textsc{Bcast}(k, msg, m)$, with the signature of the lecture.

b) Find its cost in terms of p, L, m, b and the number of pieces r.

c) Which value of r minimizes this cost, and is therefore the one the algorithm uses? What is the cost then, and what does it become for large messages? Compare with the $(p-1)(L+m\,b)$ of the lecture.

Exercise 2: the pipelined broadcast, root P_0, $r = 3$ pieces



- The BCAST of the lecture sends the whole array around the ring: $p - 1$ steps of $L + m\,b$, **one link busy at a time**.
- Pipelined: the root sends its r pieces one after the other; every other processor forwards a piece while it receives the next one, and keeps a copy; here, at step 3, **three links are busy**.

Exercise 2: Pipelined broadcast on a ring

The $\textsc{Bcast}(k, msg, m)$ of the lecture relays the whole array of the root around the ring, in $(p-1)(L+m\,b)$: again a single link is busy at a time. Pipelining applies: the root cuts its array into r pieces of m/r elements (r divides m) and sends them one after the other; every other processor forwards each piece as soon as it has it, while it receives the next one, and keeps a copy. The number of pieces r is not a parameter: the algorithm chooses it internally, as well as possible (question (c)).

a) Write the pipelined algorithm $\textsc{Bcast}(k, msg, m)$, with the signature of the lecture.

b) Find its cost in terms of p, L, m, b and the number of pieces r.

c) Which value of r minimizes this cost, and is therefore the one the algorithm uses? What is the cost then, and what does it become for large messages? Compare with the $(p-1)(L+m\,b)$ of the lecture.

Exercise 2(a): the pipelined BCAST

- A pipelined transfer from the root **all the way round the ring**, to $P_{(k-1) \bmod p}$: $d = p - 1$ links; every other processor is an intermediate processor of SENDTO, which keeps the pieces in its own *msg*: exactly what a broadcast wants.
- So BCAST(k, msg, m) is **simply** SENDTO$(k, (k-1) \bmod p, msg, m, r)$, with r computed by the algorithm itself (line 2, the formula of (c)): this is why SENDTO was written with indices modulo p.
- Unfolded: the root sends the r pieces one after the other; P_{k-1} receives them one after the other; the others receive piece 0, forward piece $t-1$ while receiving piece t, and forward the last piece.
- With $r = 1$: exactly the BCAST of the lecture.

BCAST(k, msg, m)

1: $p \leftarrow$ NUMPROCS()
2: $r \leftarrow r^\star = \sqrt{(p-2)\, m\, b / L}$ ▶ see (c)
3: SENDTO$(k, (k-1) \bmod p, msg, m, r)$

Exercise 2: Pipelined broadcast on a ring

The $\textsc{Bcast}(k, msg, m)$ of the lecture relays the whole array of the root around the ring, in $(p-1)(L + m\,b)$: again a single link is busy at a time. Pipelining applies: the root cuts its array into r pieces of m/r elements (r divides m) and sends them one after the other; every other processor forwards each piece as soon as it has it, while it receives the next one, and keeps a copy. The number of pieces r is not a parameter: the algorithm chooses it internally, as well as possible (question (c)).

a) Write the pipelined algorithm $\textsc{Bcast}(k, msg, m)$, with the signature of the lecture.

b) Find its cost in terms of p, L, m, b and the number of pieces r.

c) Which value of r minimizes this cost, and is therefore the one the algorithm uses? What is the cost then, and what does it become for large messages? Compare with the $(p-1)(L + m\,b)$ of the lecture.

Exercise 2(b): the cost of the pipelined broadcast

The message crosses $d = p - 1$ links, from P_k to P_{k-1}: by Exercise 1(b), the last piece crosses the last link at step $(p-1) + (r-1)$:

$$T_{comm} = (p + r - 2)\left(L + \frac{m}{r}\, b\right).$$

step	1	2	3	4	5	6
$P_0 \to P_1$	0	1	2			
$P_1 \to P_2$		0	1	2		
$P_2 \to P_3$			0	1	2	
$P_3 \to P_4$				0	1	2

$p = 5$, root P_0, $r = 3$ pieces: 6 steps instead of 4, but each one 3 times shorter (up to the latency); from step 3 on, all the links are busy.

Exercise 2: Pipelined broadcast on a ring

The $\textsc{Bcast}(k, msg, m)$ of the lecture relays the whole array of the root around the ring, in $(p-1)(L + m\,b)$: again a single link is busy at a time. Pipelining applies: the root cuts its array into r pieces of m/r elements (r divides m) and sends them one after the other; every other processor forwards each piece as soon as it has it, while it receives the next one, and keeps a copy. The number of pieces r is not a parameter: the algorithm chooses it internally, as well as possible (question (c)).

a) Write the pipelined algorithm $\textsc{Bcast}(k, msg, m)$, with the signature of the lecture.

b) Find its cost in terms of p, L, m, b and the number of pieces r.

c) Which value of r minimizes this cost, and is therefore the one the algorithm uses? What is the cost then, and what does it become for large messages? Compare with the $(p-1)(L + m\,b)$ of the lecture.

Exercise 2(c): the best number of pieces

- As in Exercise 1(c) with $d = p - 1$: $T_{comm} = (p-2)\,L + m\,b + r\,L + (p-2)\,m\,b/r$ is minimal for $r^\star = \sqrt{(p-2)\,m\,b/L}$, and then $T_{comm} = (p-2)\,L + m\,b + 2\sqrt{(p-2)\,L\,m\,b}$.
- Large messages: $T_{comm} \approx m\,b$, the time to push the message through **one link, independent of p**; the lecture's version costs $(p-1)(L + m\,b) \approx (p-1)\,m\,b$: the pipelined broadcast is **$p-1$ times faster**.
- Small messages: $r = 1$ is the best and nothing is gained: the $p-1$ latencies are unavoidable, the message has to cross $p-1$ links.
- This $r^\star$ is what line 2 of the algorithm computes (rounded to a divisor of m, at least 1), from p, m and the machine constants L, b.

Algorithm	T_{comm}	T_{comp}	Remark
BCAST (lecture)	$(p-1)(L + m\,b)$	0	one link busy at a time
BCAST, r pieces	$(p + r - 2)(L + m\,b/r)$	0	all the links busy after $p-1$ steps
BCAST, $r = r^\star$	$(p-2)\,L + m\,b + 2\sqrt{(p-2)\,L\,m\,b}$	0	$\approx m\,b$ for large m, whatever p

Outline

1. Exercise 1: Pipelined point-to-point communication
2. Exercise 2: Pipelined broadcast on a ring
3. Exercise 3: Reduction on a ring
4. Exercise 4: Scan on a ring
5. Exercise 5: Matrix–vector product on a ring
6. Exercise 6: Scatter on a ring
7. Exercise 7: Transposition on a ring
8. Exercise 8: Grids, hypercubes and binary trees

Exercise 3: Reduction on a ring

REDUCE$(k, msg, m, \otimes)$: every processor holds m elements in its array *msg*; at the end, the root P_k holds in *msg* their combination, element by element, with the associative and commutative operation $\otimes$ (sum, max, ...). The lecture gave its principle: a partial result travels around the ring from P_{k+1} to P_k, every processor combining what it receives with its own elements before forwarding it. Combining s received elements, held in a buffer *tmp*, into s elements of *msg* is a loop $msg[a+l] \leftarrow tmp[l] \otimes msg[a+l]$, $l = 0, \ldots, s-1$, which costs $s\tau$.

a) Write the non-pipelined algorithm REDUCE$(k, msg, m, \otimes)$, with the structure of the BCAST of the lecture.

b) Find its communication cost T_{comm} and its computation cost T_{comp}.

c) Pipeline it with r pieces of m/r elements, the number of pieces being chosen internally by the algorithm (question (e)): write the pipelined REDUCE$(k, msg, m, \otimes)$, with the same signature. Be careful: a piece can be forwarded only once it has been received *and* combined.

d) Find its costs T_{comm} and T_{comp} (draw what each processor does over time).

e) Which value of r minimizes the total cost? This is the value the algorithm uses.

Exercise 3: Reduction on a ring

REDUCE$(k, msg, m, \otimes)$: every processor holds m elements in its array *msg*; at the end, the root P_k holds in *msg* their combination, element by element, with the associative and commutative operation $\otimes$ (sum, max, ...). The lecture gave its principle: a partial result travels around the ring from P_{k+1} to P_k, every processor combining what it receives with its own elements before forwarding it. Combining s received elements, held in a buffer *tmp*, into s elements of *msg* is a loop $msg[a+l] \leftarrow tmp[l] \otimes msg[a+l]$, $l = 0, \ldots, s-1$, which costs $s\tau$.

a) Write the non-pipelined algorithm REDUCE$(k, msg, m, \otimes)$, with the structure of the BCAST of the lecture.

b) Find its communication cost T_{comm} and its computation cost T_{comp}.

c) Pipeline it with r pieces of m/r elements, the number of pieces being chosen internally by the algorithm (question (e)): write the pipelined REDUCE$(k, msg, m, \otimes)$, with the same signature. Be careful: a piece can be forwarded only once it has been received *and* combined.

d) Find its costs T_{comm} and T_{comp} (draw what each processor does over time).

e) Which value of r minimizes the total cost? This is the value the algorithm uses.

Exercise 3(a): REDUCE, the non-pipelined version

- The chain **starts at** P_{k+1}, which just sends its array (line 3), and **ends at the root** (lines 5–6); the others receive the partial result in *tmp* (line 8), combine it into their array element by element (line 9) and forward it (line 10).
- With x_i the array of P_i: after its loop, P_q holds $x_{k+1} \otimes \cdots \otimes x_q$; the root ends with $x_{k+1} \otimes \cdots \otimes x_k$ ($\otimes$ commutative: the order does not matter).
- Compared with BCAST: root and neighbours **mirrored** (the chain enters the root instead of leaving it), and the combination loop sits between the RECV and the SEND.

REDUCE$(k, msg, m, \otimes)$

```
1:  q ← MyRank(); p ← NumProcs()
2:  if q = (k + 1) mod p then
3:    Send(msg, m)
4:  else if q = k then
5:    Recv(tmp, m)
6:    for l = 0, ..., m − 1 do msg[l] ← tmp[l] ⊗ msg[l]
7:  else
8:    Recv(tmp, m)
9:    for l = 0, ..., m − 1 do msg[l] ← tmp[l] ⊗ msg[l]
10:   Send(msg, m)
```

Exercise 3: Reduction on a ring

REDUCE$(k, msg, m, \otimes)$: every processor holds m elements in its array *msg*; at the end, the root P_k holds in *msg* their combination, element by element, with the associative and commutative operation $\otimes$ (sum, max, ...). The lecture gave its principle: a partial result travels around the ring from P_{k+1} to P_k, every processor combining what it receives with its own elements before forwarding it. Combining s received elements, held in a buffer *tmp*, into s elements of *msg* is a loop $msg[a+l] \leftarrow tmp[l] \otimes msg[a+l]$, $l = 0, \ldots, s-1$, which costs $s\tau$.

a) Write the non-pipelined algorithm REDUCE$(k, msg, m, \otimes)$, with the structure of the BCAST of the lecture.

b) Find its communication cost T_{comm} and its computation cost T_{comp}.

c) Pipeline it with r pieces of m/r elements, the number of pieces being chosen internally by the algorithm (question (e)): write the pipelined REDUCE$(k, msg, m, \otimes)$, with the same signature. Be careful: a piece can be forwarded only once it has been received *and* combined.

d) Find its costs T_{comm} and T_{comp} (draw what each processor does over time).

e) Which value of r minimizes the total cost? This is the value the algorithm uses.

Exercise 3(b): its costs

- $p-1$ messages of m elements travel one after the other, one link busy at a time: $T_{comm} = (p-1)(L + m\,b)$.
- The $p-1$ combination loops, of m operations each, also take place one after the other, each one waiting for the previous message: $T_{comp} = (p-1)\,m\,\tau$.
- Total time $(p-1)(L + m\,b + m\,\tau)$: nothing overlaps, neither the messages nor the combinations.

REDUCE$(k, msg, m, \otimes)$

```
1:  q ← MyRank(); p ← NumProcs()
2:  if q = (k + 1) mod p then
3:    Send(msg, m)
4:  else if q = k then
5:    Recv(tmp, m)
6:    for l = 0, ..., m − 1 do msg[l] ← tmp[l] ⊗ msg[l]
7:  else
8:    Recv(tmp, m)
9:    for l = 0, ..., m − 1 do msg[l] ← tmp[l] ⊗ msg[l]
10:   Send(msg, m)
```

Algorithm	T_{comm}	T_{comp}	Remark
REDUCE	$(p-1)(L + m\,b)$	$(p-1)\,m\,\tau$	nothing overlaps

Exercise 3: Reduction on a ring

REDUCE$(k, msg, m, \otimes)$: every processor holds m elements in its array *msg*; at the end, the root P_k holds in *msg* their combination, element by element, with the associative and commutative operation $\otimes$ (sum, max, ...). The lecture gave its principle: a partial result travels around the ring from P_{k+1} to P_k, every processor combining what it receives with its own elements before forwarding it. Combining s received elements, held in a buffer *tmp*, into s elements of *msg* is a loop $msg[a+l] \leftarrow tmp[l] \otimes msg[a+l]$, $l = 0, \ldots, s-1$, which costs $s\tau$.

a) Write the non-pipelined algorithm REDUCE$(k, msg, m, \otimes)$, with the structure of the BCAST of the lecture.

b) Find its communication cost T_{comm} and its computation cost T_{comp}.

c) Pipeline it with r pieces of m/r elements, the number of pieces being chosen internally by the algorithm (question (e)): write the pipelined REDUCE$(k, msg, m, \otimes)$, with the same signature. Be careful: a piece can be forwarded only once it has been received *and* combined.

d) Find its costs T_{comm} and T_{comp} (draw what each processor does over time).

e) Which value of r minimizes the total cost? This is the value the algorithm uses.

Exercise 3(c): the pipelined REDUCE

- r pieces of $s = m/r$ elements, r the $r^\star$ of (e) (line 2); piece t goes into $msg[t\,s \ldots (t+1)s-1]$. P_{k+1} sends its pieces (line 4); the root receives and combines them (lines 6–8).
- The others receive and combine piece 0 (lines 10–11), then **forward piece $t-1$ while receiving piece t** (line 13) and combine piece t (line 14); the last piece leaves at line 15.
- One buffer *tmp* is enough: its piece is consumed before the next RECV; the piece being sent lives in *msg*.

REDUCE$(k, msg, m, \otimes)$, PIPELINED

```
1:  q ← MyRank(); p ← NumProcs()
2:  r ← r* (see (e)); s ← m/r
3:  if q = (k + 1) mod p then
4:    for t = 0, ..., r − 1 do Send(msg[t s ...], s)
5:  else if q = k then
6:    for t = 0, ..., r − 1 do
7:      Recv(tmp, s)
8:      for l = 0, ..., s − 1 do msg[t s + l] ← tmp[l] ⊗ msg[t s + l]
9:  else
10:   Recv(tmp, s)
11:   for l = 0, ..., s − 1 do msg[l] ← tmp[l] ⊗ msg[l]
12:   for t = 1, ..., r − 1 do
13:     Send(msg[(t − 1)s ...], s) ‖ Recv(tmp, s)
14:     for l = 0, ..., s − 1 do msg[t s + l] ← tmp[l] ⊗ msg[t s + l]
15:   Send(msg[(r − 1)s ...], s)
```

Exercise 3: Reduction on a ring

REDUCE$(k, msg, m, \otimes)$: every processor holds m elements in its array *msg*; at the end, the root P_k holds in *msg* their combination, element by element, with the associative and commutative operation $\otimes$ (sum, max, ...). The lecture gave its principle: a partial result travels around the ring from P_{k+1} to P_k, every processor combining what it receives with its own elements before forwarding it. Combining s received elements, held in a buffer *tmp*, into s elements of *msg* is a loop $msg[a+l] \leftarrow tmp[l] \otimes msg[a+l]$, $l = 0, \ldots, s-1$, which costs $s\tau$.

a) Write the non-pipelined algorithm REDUCE$(k, msg, m, \otimes)$, with the structure of the BCAST of the lecture.

b) Find its communication cost T_{comm} and its computation cost T_{comp}.

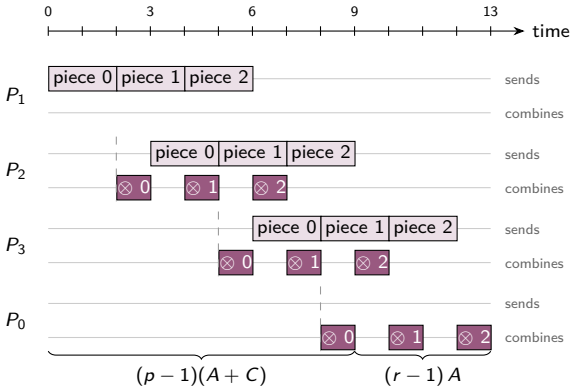
c) Pipeline it with r pieces of m/r elements, the number of pieces being chosen internally by the algorithm (question (e)): write the pipelined REDUCE$(k, msg, m, \otimes)$, with the same signature. Be careful: a piece can be forwarded only once it has been received *and* combined.

d) Find its costs T_{comm} and T_{comp} (draw what each processor does over time).

e) Which value of r minimizes the total cost? This is the value the algorithm uses.

Exercise 3(d): what every processor does over time

$A = L + (m/r)\,b$: one piece on one link; $C = (m/r)\,\tau$: the combination of one piece; $p = 4$, root P_0, $r = 3$. P_1 sends its pieces back to back; on every other processor, piece t is received, combined, then forwarded.



$A = L + (m/r)\,b$ (2 units), $C = (m/r)\,\tau$ (1 unit); the dashed lines mark the arrival of piece 0

Exercise 3(d): the costs of the pipelined REDUCE

- Let h be the position of a processor in the chain ($h = 0$ for P_{k+1}, $h = p-2$ for P_{k-1}). Piece t leaves the processor at position h at time $h\,(A+C) + t\,A$: each hop adds a transfer and a combination, $A + C$; each piece adds a transfer, A (the combination of piece t ends exactly when the transfer of piece $t-1$ ends).
- The root receives the last piece at time $(p-2)(A+C) + (r-1)\,A + A$ and combines it in C: $T = (p-1)(A+C) + (r-1)\,A$, that is

$$T_{comm} = (p+r-2)\left(L + \frac{m}{r}\,b\right), \qquad T_{comp} = (p-1)\,\frac{m}{r}\,\tau.$$

- The communication part is that of the pipelined BCAST; the computation part is **r times smaller** than in (b): the combinations of the different processors overlap, only one chain of $p-1$ of them, each r times shorter, is on the critical path. The work is unchanged, it is spread over the pipeline.

Algorithm	T_{comm}	T_{comp}	Remark
REDUCE	$(p-1)(L + m\,b)$	$(p-1)\,m\,\tau$	nothing overlaps
REDUCE, r pieces	$(p+r-2)(L + m\,b/r)$	$(p-1)\,m\,\tau/r$	messages and combinations overlap

Exercise 3: Reduction on a ring

REDUCE$(k, msg, m, \otimes)$: every processor holds m elements in its array *msg*; at the end, the root P_k holds in *msg* their combination, element by element, with the associative and commutative operation $\otimes$ (sum, max, ...). The lecture gave its principle: a partial result travels around the ring from P_{k+1} to P_k, every processor combining what it receives with its own elements before forwarding it. Combining s received elements, held in a buffer *tmp*, into s elements of *msg* is a loop $msg[a+l] \leftarrow tmp[l] \otimes msg[a+l]$, $l = 0, \ldots, s-1$, which costs $s\tau$.

a) Write the non-pipelined algorithm REDUCE$(k, msg, m, \otimes)$, with the structure of the BCAST of the lecture.

b) Find its communication cost T_{comm} and its computation cost T_{comp}.

c) Pipeline it with r pieces of m/r elements, the number of pieces being chosen internally by the algorithm (question (e)): write the pipelined REDUCE$(k, msg, m, \otimes)$, with the same signature. Be careful: a piece can be forwarded only once it has been received *and* combined.

d) Find its costs T_{comm} and T_{comp} (draw what each processor does over time).

e) Which value of r minimizes the total cost? This is the value the algorithm uses.

Exercise 3(e): the best number of pieces

université PARIS-SACLAY

- The total time of (d) as a function of r, with $K = (p-2)\,m\,b + (p-1)\,m\,\tau$; the first two terms do not depend on r:

$$T(r) = (p+r-2)\left(L + \frac{m}{r}\,b\right) + (p-1)\,\frac{m}{r}\,\tau = (p-2)\,L + m\,b + r\,L + \frac{K}{r}.$$

- Derivative: $T'(r) = L - K/r^2$; $\quad T'(r) = 0 \iff r^2 = K/L \iff r^\star = \sqrt{K/L}$ (rounded to a divisor of m):

$$r^\star = \sqrt{\frac{m\left((p-2)\,b + (p-1)\,\tau\right)}{L}} \approx \sqrt{\frac{(p-2)\,m\,(b+\tau)}{L}}.$$

- Second derivative: $T''(r) = 2K/r^3 > 0$ for every $r > 0$: T is **convex**, $r^\star$ is a **minimum**, not a maximum.
- At $r^\star$, $r\,L = K/r = \sqrt{L\,K}$, so

$$T(r^\star) = (p-2)\,L + m\,b + 2\sqrt{L\,K} = (p-2)\,L + m\,b + 2\sqrt{L\,m\left((p-2)\,b + (p-1)\,\tau\right)}.$$

Exercise 3(e): the optimum and what it means

- $T(r^\star) = (p-2)\,L + m\,b + 2\sqrt{L\,m\,\big((p-2)\,b + (p-1)\,\tau\big)}$, for $r^\star = \sqrt{m\,\big((p-2)\,b + (p-1)\,\tau\big)/L}$.
- Compared with the pipelined BCAST of 2(c): the same formula with b replaced by $b+\tau$ in the square root (up to the factors $p-2$ and $p-1$), the price of the combinations.
- Large messages: $T \approx m\,b$ again: the computation **disappears from the critical path** as well, as the term in τ only grows like $\sqrt{m}$.
- This $r^\star$ is what line 2 of the algorithm computes, from p, m and the machine constants L, b, τ.

Algorithm	T_{comm}	T_{comp}	Remark
REDUCE, $r = r^\star$	$(p-2)\,L + m\,b + 2\sqrt{L\,m\,((p-2)\,b + (p-1)\,\tau)}$ in total		$\approx m\,b$ for large m

Outline

1. Exercise 1: Pipelined point-to-point communication
2. Exercise 2: Pipelined broadcast on a ring
3. Exercise 3: Reduction on a ring
4. Exercise 4: Scan on a ring
5. Exercise 5: Matrix–vector product on a ring
6. Exercise 6: Scatter on a ring
7. Exercise 7: Transposition on a ring
8. Exercise 8: Grids, hypercubes and binary trees

Exercise 4: Scan on a ring

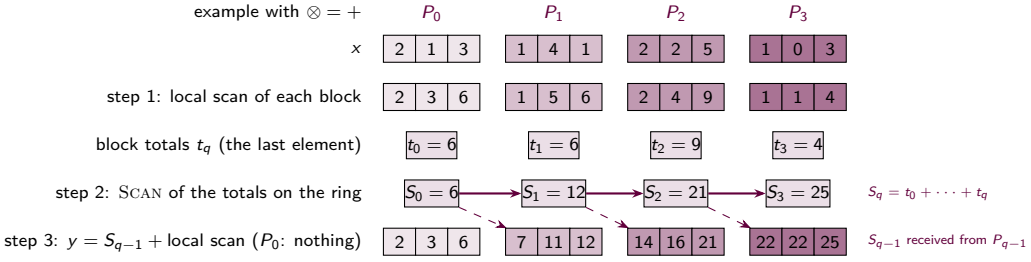
An array x of N elements is distributed by blocks of $n = N/p$ (p divides N): P_q holds the elements $x[q\,n], \ldots, x[(q+1)n-1]$ in a local array $x[0 \ldots n-1]$. We want its *scan* for an associative operation $\otimes$ (sum, max, ...), as in the PRAM session: $y[i] = x[0] \otimes x[1] \otimes \cdots \otimes x[i]$ for $i = 0, \ldots, N-1$, distributed in the same way: P_q must compute $y[q\,n], \ldots, y[(q+1)n-1]$ in a local array $y[0 \ldots n-1]$. We follow the ScanP of the PRAM session: every processor scans its own block; the p block totals are scanned across the processors; every processor then finishes its block with the result.

a) Write Scan$(v, \otimes)$ on the ring: every processor holds one value v; at the end, P_q holds $v_0 \otimes v_1 \otimes \cdots \otimes v_q$ in v (v_i: the initial value of P_i). Find its costs T_{comm} and T_{comp}. Can it be pipelined like Bcast? Can any algorithm do better on the unidirectional ring?

b) Write ScanP$(x, y, n, \otimes)$, which first scans the local block, then calls Scan on the block totals, and finally finishes the block with a last update. Which quantity does this update combine with the elements of P_q, and how does P_q obtain it? Find the costs T_{comm} and T_{comp} of ScanP.

c) The sequential scan costs $(N-1)\,\tau \approx N\,\tau$. What are the speedup and the efficiency of ScanP? What is the limit of the efficiency for large N, and why is it not 1?

Exercise 4: the three steps on an example, $\otimes = +$



- $p = 4$, $n = 3$. Step 1: every processor scans its own block and gets its total t_q; step 2: the totals are scanned across the processors, $S_q = t_0 \otimes \cdots \otimes t_q$; step 3: every processor combines S_{q-1}, the total of the blocks before it, with its whole block (on the left).
- The structure of the SCANP of the PRAM session; only step 2 communicates.

Exercise 4: Scan on a ring

An array x of N elements is distributed by blocks of $n = N/p$ (p divides N): P_q holds the elements $x[q\,n], \ldots, x[(q+1)n-1]$ in a local array $x[0 \ldots n-1]$. We want its *scan* for an associative operation $\otimes$ (sum, max, ...), as in the PRAM session: $y[i] = x[0] \otimes x[1] \otimes \cdots \otimes x[i]$ for $i = 0, \ldots, N-1$, distributed in the same way: P_q must compute $y[q\,n], \ldots, y[(q+1)n-1]$ in a local array $y[0 \ldots n-1]$. We follow the ScanP of the PRAM session: every processor scans its own block; the p block totals are scanned across the processors; every processor then finishes its block with the result.

a) Write Scan$(v, \otimes)$ on the ring: every processor holds one value v; at the end, P_q holds $v_0 \otimes v_1 \otimes \cdots \otimes v_q$ in v (v_i: the initial value of P_i). Find its costs T_{comm} and T_{comp}. Can it be pipelined like Bcast? Can any algorithm do better on the unidirectional ring?

b) Write ScanP$(x, y, n, \otimes)$, which first scans the local block, then calls Scan on the block totals, and finally finishes the block with a last update. Which quantity does this update combine with the elements of P_q, and how does P_q obtain it? Find the costs T_{comm} and T_{comp} of ScanP.

c) The sequential scan costs $(N-1)\,\tau \approx N\,\tau$. What are the speedup and the efficiency of ScanP? What is the limit of the efficiency for large N, and why is it not 1?

Exercise 4(a): SCAN of one value per processor

- The chain of REDUCE, but every processor **keeps the partial result**: P_0 sends v_0 (line 3); P_q receives $v_0 \otimes \cdots \otimes v_{q-1}$ in *tmp* (line 5), combines its value on the right (line 6: $\otimes$ need not be commutative) and forwards the result unless it is the last processor (line 7).
- $p-1$ messages of one element, one after the other, and $p-1$ operations, one after the other: $T_{comm} = (p-1)(L+b)$, $T_{comp} = (p-1)\tau$; one link busy at a time.
- **No pipelining**: one element per message, nothing to cut. **No algorithm does better**: the value of P_0 must reach P_{p-1}, $p-1$ links away: $p-1$ messages one after the other at least.

SCAN$(v, \otimes)$

```
1: q ← MyRank(); p ← NumProcs()
2: if q = 0 then
3:    Send(v, 1)
4: else
5:    Recv(tmp, 1)              ▶ v_0 ⊗ ··· ⊗ v_{q-1}
6:    v ← tmp ⊗ v               ▶ v_0 ⊗ ··· ⊗ v_q
7:    if q < p − 1 then Send(v, 1)
```

Algorithm	T_{comm}	T_{comp}	Remark
SCAN$(v, \otimes)$	$(p-1)(L+b)$	$(p-1)\tau$	a chain, one link busy at a time; no pipelining possible

Exercise 4: Scan on a ring

An array x of N elements is distributed by blocks of $n = N/p$ (p divides N): P_q holds the elements $x[q\,n], \ldots, x[(q+1)n-1]$ in a local array $x[0 \ldots n-1]$. We want its *scan* for an associative operation $\otimes$ (sum, max, ...), as in the PRAM session: $y[i] = x[0] \otimes x[1] \otimes \cdots \otimes x[i]$ for $i = 0, \ldots, N-1$, distributed in the same way: P_q must compute $y[q\,n], \ldots, y[(q+1)n-1]$ in a local array $y[0 \ldots n-1]$. We follow the ScanP of the PRAM session: every processor scans its own block; the p block totals are scanned across the processors; every processor then finishes its block with the result.

a) Write Scan$(v, \otimes)$ on the ring: every processor holds one value v; at the end, P_q holds $v_0 \otimes v_1 \otimes \cdots \otimes v_q$ in v (v_i: the initial value of P_i). Find its costs T_{comm} and T_{comp}. Can it be pipelined like Bcast? Can any algorithm do better on the unidirectional ring?

b) Write ScanP$(x, y, n, \otimes)$, which first scans the local block, then calls Scan on the block totals, and finally finishes the block with a last update. Which quantity does this update combine with the elements of P_q, and how does P_q obtain it? Find the costs T_{comm} and T_{comp} of ScanP.

c) The sequential scan costs $(N-1)\,\tau \approx N\,\tau$. What are the speedup and the efficiency of ScanP? What is the limit of the efficiency for large N, and why is it not 1?

Exercise 4(b): ScanP, the three steps

université PARIS-SACLAY

- Step 1, the local scan (lines 2–3): $y[j]$ holds $x[q\,n] \otimes \cdots \otimes x[q\,n+j]$, **its own block only**; missing, on the left: the blocks before, $o_q = t_0 \otimes \cdots \otimes t_{q-1}$, a **single value** per processor, contributed by $P_0, \ldots, P_{q-1}$.
- Step 2: $t_q = y[n-1]$ (line 4) and Scan on the totals (line 5) give $t = S_q = t_0 \otimes \cdots \otimes t_q$; P_q needs $o_q = S_{q-1}$, the value of its left neighbour: the ScanP of TD1 read $S'[j-1]$ in the shared memory; on the ring it has to **travel**: one more exchange (lines 6–7). With $+$, $S_q - t_q$ would do; a general $\otimes$ has no inverse.
- Step 3, the final update (line 8, nothing on P_0): $y[j] = o_q \otimes x[q\,n] \otimes \cdots \otimes x[q\,n+j]$, the prefix wanted.
- Scan plus one exchange: $T_{comm} = p\,(L+b)$; $(n-1)+(p-1)+n$ operations: $T_{comp} = (2n+p-2)\,\tau$. (Scan could return S_{q-1} and save the exchange.)

ScanP$(x, y, n, \otimes)$

1: $q \leftarrow$ MyRank(); $p \leftarrow$ NumProcs()
2: $y[0] \leftarrow x[0]$
3: **for** $j = 1, \ldots, n-1$ **do** $y[j] \leftarrow y[j-1] \otimes x[j]$
4: $t \leftarrow y[n-1]$ ▶ the block total t_q
5: Scan$(t, \otimes)$ ▶ $t = S_q = t_0 \otimes \cdots \otimes t_q$
6: **if** $q < p-1$ **then** Send$(t, 1)$ ▶ S_q to P_{q+1}
7: **if** $q > 0$ **then** Recv$(o, 1)$ ▶ $o = S_{q-1} = o_q$
8: **if** $q > 0$ **then for** $j = 0, \ldots, n-1$ **do** $y[j] \leftarrow o \otimes y[j]$

Exercise 4: Scan on a ring

An array x of N elements is distributed by blocks of $n = N/p$ (p divides N): P_q holds the elements $x[q\,n], \ldots, x[(q+1)n-1]$ in a local array $x[0 \ldots n-1]$. We want its *scan* for an associative operation $\otimes$ (sum, max, ...), as in the PRAM session: $y[i] = x[0] \otimes x[1] \otimes \cdots \otimes x[i]$ for $i = 0, \ldots, N-1$, distributed in the same way: P_q must compute $y[q\,n], \ldots, y[(q+1)n-1]$ in a local array $y[0 \ldots n-1]$. We follow the ScanP of the PRAM session: every processor scans its own block; the p block totals are scanned across the processors; every processor then finishes its block with the result.

a) Write Scan$(v, \otimes)$ on the ring: every processor holds one value v; at the end, P_q holds $v_0 \otimes v_1 \otimes \cdots \otimes v_q$ in v (v_i: the initial value of P_i). Find its costs T_{comm} and T_{comp}. Can it be pipelined like Bcast? Can any algorithm do better on the unidirectional ring?

b) Write ScanP$(x, y, n, \otimes)$, which first scans the local block, then calls Scan on the block totals, and finally finishes the block with a last update. Which quantity does this update combine with the elements of P_q, and how does P_q obtain it? Find the costs T_{comm} and T_{comp} of ScanP.

c) The sequential scan costs $(N-1)\,\tau \approx N\,\tau$. What are the speedup and the efficiency of ScanP? What is the limit of the efficiency for large N, and why is it not 1?

Exercise 4(c): speedup and efficiency

- $T \approx (2n+p)\,\tau + p\,(L+b)$ with $n = N/p$, and $T_{seq} = (N-1)\,\tau \approx N\,\tau$:

$$S = \frac{T_{seq}}{T} \approx \frac{N}{2N/p + p + p\,(L+b)/\tau}, \qquad E = \frac{S}{p} \approx \frac{1}{2 + \dfrac{p^2}{N} + \dfrac{p^2(L+b)}{N\,\tau}}.$$

- When N grows ($N \gg p^2(L+b)/\tau$), the two last terms vanish: $E \to 1/2$, the speedup tends to $p/2$, not to p.
- Not because of the communication, which is amortized, but because of the **work**: every processor goes over its block twice, to scan it and to update it, $2n$ operations instead of n: $2N$ operations in total against $N-1$ sequentially.
- Unavoidable here: the offset of a block is known only once the totals have been scanned (total first, then one scan pass with the offset as the initial value: $2n$ operations as well).

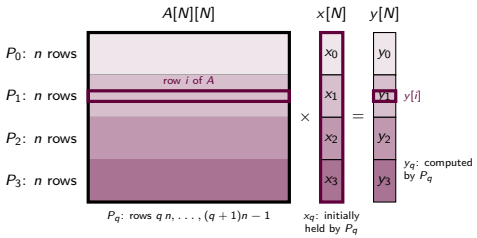
Algorithm	T_{comm}	T_{comp}	Remark
$\textsc{Scan}(v, \otimes)$	$(p-1)(L+b)$	$(p-1)\,\tau$	a chain, no pipelining possible
ScanP	$p\,(L+b)$	$(2n+p-2)\,\tau$	$E \to 1/2$: two passes over the block

Outline

1. Exercise 1: Pipelined point-to-point communication
2. Exercise 2: Pipelined broadcast on a ring
3. Exercise 3: Reduction on a ring
4. Exercise 4: Scan on a ring
5. Exercise 5: Matrix–vector product on a ring
6. Exercise 6: Scatter on a ring
7. Exercise 7: Transposition on a ring
8. Exercise 8: Grids, hypercubes and binary trees

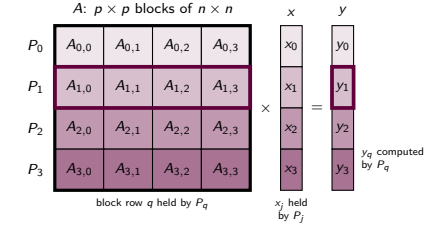
Exercise 5: Matrix–vector product on a ring

Let A be an $N \times N$ matrix and x a vector of N elements; we want to compute $y = A\,x$, that is $y[i] = \sum_{j=0}^{N-1} A[i][j]\,x[j]$. The data are distributed by blocks of $n = N/p$ (p divides N): P_q holds the rows $q\,n, \ldots, (q+1)n-1$ of A in a local array $A[0 \ldots n-1][0 \ldots N-1]$, and the elements $x[q\,n], \ldots, x[(q+1)n-1]$ of x in a local array $x[0 \ldots n-1]$; it must compute the elements $y[q\,n], \ldots, y[(q+1)n-1]$ of y in a local array $y[0 \ldots n-1]$.



Exercise 5: Matrix–vector product on a ring (cont.)

Seen by blocks: cut A into $p \times p$ blocks of $n \times n$ elements, $A_{q,j}$ being the block at the rows of P_q and the columns $j\,n, \ldots, (j+1)n-1$, and cut x and y into p blocks of n elements, x_j and y_j. Then $y_q = \sum_{j=0}^{p-1} A_{q,j}\, x_j$: the block y_q of P_q is the sum of p small matrix–vector products, one per block of x, each one an $n \times n$ block times n elements. P_q holds the whole block row $A_{q,0}, \ldots, A_{q,p-1}$ of A, but a single block of x, its own x_q.



Exercise 5: Matrix–vector product on a ring

a) Which elements of x does P_q need to compute its block of y? Which processors hold them? How many phases of computation and of communication must take place, and what should be computed and communicated in each phase?

b) Design $\textsc{MatVec}(A, x, y, n)$ on the ring, where A is the local array of n rows and $N = n\,p$ columns of the processor, and x and y its local arrays of n elements: the blocks of x travel around the ring; at every step, every processor uses the block it holds, then passes it to its right neighbour.

c) Find its communication cost and its computation cost.

d) The sequential product costs $2N^2\,\tau$. What are the speedup and the efficiency of $\textsc{MatVec}$? When is the efficiency close to 1?

Exercise 5: Matrix–vector product on a ring

a) Which elements of x does P_q need to compute its block of y? Which processors hold them? How many phases of computation and of communication must take place, and what should be computed and communicated in each phase?

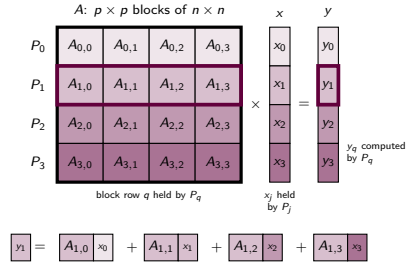
b) Design $\textsc{MatVec}(A, x, y, n)$ on the ring, where A is the local array of n rows and $N = n\,p$ columns of the processor, and x and y its local arrays of n elements: the blocks of x travel around the ring; at every step, every processor uses the block it holds, then passes it to its right neighbour.

c) Find its communication cost and its computation cost.

d) The sequential product costs $2N^2\,\tau$. What are the speedup and the efficiency of $\textsc{MatVec}$? When is the efficiency close to 1?

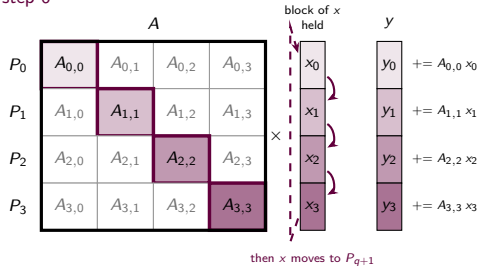
Exercise 5(a): what P_q needs, and the phases

- $y[i] = \sum_{j=0}^{N-1} A[i][j]\, x[j]$ uses the **whole vector** x, for every row i: P_q needs all the p blocks of x and holds only its own; x_j is held by P_j.
- By blocks, $y_q = \sum_{j=0}^{p-1} A_{q,j}\, x_j$: p **computation phases**, one per block of x; in each, P_q computes one block product $A_{q,j}\, x_j$ with the block it holds and adds it to y_q.
- Between two of them, a **communication phase**: every block of x moves to the next processor, P_q sends the block it has just used to P_{q+1} and receives a new one from P_{q-1}: $p-1$ communication phases, after which every block has visited every processor.

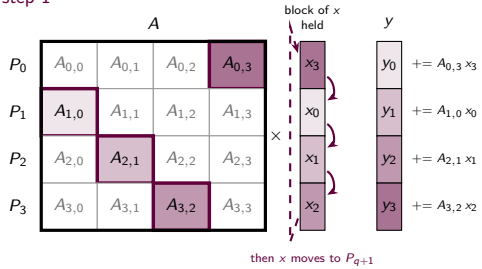


Exercise 5(a): the phases, steps 0 and 1 ($p = 4$)

step 0



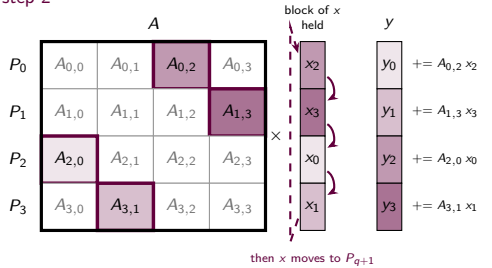
step 1



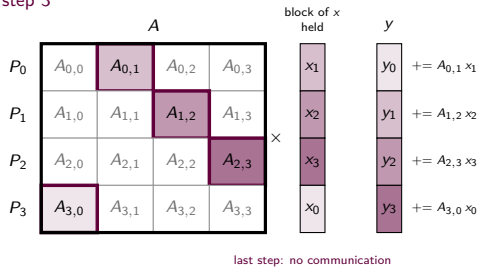
- Step s: P_q holds the block x_j with $j = (q - s) \bmod p$, uses the block $A_{q,j}$ of its rows (filled in the colour of x_j) and adds $A_{q,j}\, x_j$ to its y_q: the diagonal blocks at step 0, the first sub-diagonal at step 1.
- Then every block of x moves to the right neighbour (arrows): P_q sends x_j to P_{q+1} and receives x_{j-1} from P_{q-1}.

Exercise 5(a): the phases, steps 2 and 3 ($p = 4$)

step 2



step 3



- Steps 2 and 3: the remaining sub-diagonals, wrapping around; after step 3, every block of A has been used exactly once and every y_q holds its p block products: $y_q = \sum_j A_{q,j}\, x_j$.
- No communication after the last step: p computation phases and $p-1$ communication phases, with the same load on every processor at every step ($2n^2$ operations).

Exercise 5: Matrix–vector product on a ring

a) Which elements of x does P_q need to compute its block of y? Which processors hold them? How many phases of computation and of communication must take place, and what should be computed and communicated in each phase?

b) Design $\textsc{MatVec}(A, x, y, n)$ on the ring, where A is the local array of n rows and $N = n\,p$ columns of the processor, and x and y its local arrays of n elements: the blocks of x travel around the ring; at every step, every processor uses the block it holds, then passes it to its right neighbour.

c) Find its communication cost and its computation cost.

d) The sequential product costs $2N^2\,\tau$. What are the speedup and the efficiency of $\textsc{MatVec}$? When is the efficiency close to 1?

Exercise 5(b): MatVec, the blocks of x rotate

- At step s ($s = 0, \ldots, p-1$), P_q holds the block $j = (q - s) \bmod p$ of x: its own at step 0, its left neighbour's at step 1, and so on (line 5).
- It multiplies $A_{q,j}$ by it and accumulates into y (lines 6–8), then passes the block to its right neighbour while it receives the next one from its left neighbour (lines 9–11): all the blocks **rotate together**, one position per step; after $p-1$ rotations every processor has used every block.
- Two buffers, swapped before the exchange (line 10): the block sent is not overwritten by the one arriving; x itself is never modified.

MatVec(A, x, y, n)

```
1:  q ← MyRank(); p ← NumProcs()
2:  for i = 0, ..., n − 1 do y[i] ← 0
3:  tempR[0 ... n − 1] ← x[0 ... n − 1]
4:  for s = 0, ..., p − 1 do
5:    j ← (q − s) mod p                    ▶ block of x held
6:    for i = 0, ..., n − 1 do
7:      for l = 0, ..., n − 1 do
8:        y[i] ← y[i] + A[i][j n + l] · tempR[l]
9:    if s < p − 1 then
10:     tempS ↔ tempR
11:     Send(tempS, n) ∥ Recv(tempR, n)
```

Exercise 5: Matrix–vector product on a ring

a) Which elements of x does P_q need to compute its block of y? Which processors hold them? How many phases of computation and of communication must take place, and what should be computed and communicated in each phase?

b) Design $\textsc{MatVec}(A, x, y, n)$ on the ring, where A is the local array of n rows and $N = n\,p$ columns of the processor, and x and y its local arrays of n elements: the blocks of x travel around the ring; at every step, every processor uses the block it holds, then passes it to its right neighbour.

c) Find its communication cost and its computation cost.

d) The sequential product costs $2N^2\,\tau$. What are the speedup and the efficiency of $\textsc{MatVec}$? When is the efficiency close to 1?

Exercise 5(c): the costs of MatVec

- **Communication**: $p-1$ exchanges of a block of n elements, all the links busy at every exchange: $T_{comm} = (p-1)(L + n\,b) = (p-1)\big(L + \frac{N}{p}\,b\big)$.
- **Computation**: at every step, n rows times n elements, n^2 multiplications and n^2 additions; p steps: $T_{comp} = 2\,p\,n^2\,\tau = \frac{2N^2}{p}\,\tau$, the sequential work divided by p.
- The communication grows like N, the computation like N^2: the latter wins for large N.

Algorithm	T_{comm}	T_{comp}	Remark
MatVec	$(p-1)(L + N\,b/p)$	$2N^2\,\tau/p$	all the links busy at every exchange

Exercise 5(d): speedup and efficiency

- $T = T_{comp} + T_{comm} = \frac{2N^2}{p}\,\tau + (p-1)\left(L + \frac{N}{p}\,b\right)$ and $T_{seq} = 2N^2\,\tau$:

$$S = \frac{T_{seq}}{T} = \frac{2N^2\,\tau}{\dfrac{2N^2}{p}\,\tau + (p-1)\,L + (p-1)\,\dfrac{N}{p}\,b};$$

- dividing the numerator and the denominator by the computation time $\frac{2N^2}{p}\,\tau$:

$$S = \frac{p}{1 + \dfrac{p\,(p-1)\,L}{2N^2\,\tau} + \dfrac{(p-1)\,b}{2N\,\tau}}, \qquad E = \frac{S}{p} = \frac{1}{1 + \dfrac{p\,(p-1)\,L}{2N^2\,\tau} + \dfrac{(p-1)\,b}{2N\,\tau}}.$$

- Computation in N^2, communication in N: $E \to 1$ **when N grows**, once $N^2 \gg p^2 L/\tau$ and $N \gg p\,b/\tau$; with $L \approx 10^4\,\tau$, $b \approx 10\,\tau$: $N \gg 100\,p$. For a fixed N, E drops when p grows: fewer rows per processor, $p-1$ latencies paid in full.

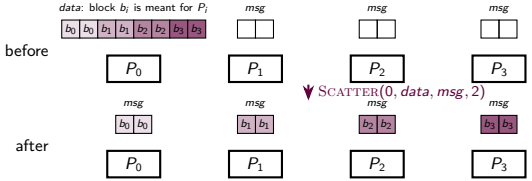
Algorithm	T_{comm}	T_{comp}	Remark
MatVec	$(p-1)(L + N\,b/p)$	$2N^2\,\tau/p$	$E \to 1$ for $N \gg 100\,p$

Outline

1. Exercise 1: Pipelined point-to-point communication
2. Exercise 2: Pipelined broadcast on a ring
3. Exercise 3: Reduction on a ring
4. Exercise 4: Scan on a ring
5. Exercise 5: Matrix–vector product on a ring
6. Exercise 6: Scatter on a ring
7. Exercise 7: Transposition on a ring
8. Exercise 8: Grids, hypercubes and binary trees

Exercise 6: Scatter on a ring

In many applications, the data are initially held by a single processor (the one that read them from a file, say) and must be distributed in equal parts to all the processors, instead of starting from an already distributed configuration. SCATTER$(k, data, msg, m)$ does exactly this: the root P_k holds an array *data* of $p\,m$ elements, made of p consecutive blocks of m elements, the block $b_i = data[i\,m \ldots (i+1)m-1]$ being meant for P_i; at the end, every P_i holds its block b_i in its array *msg* of m elements.

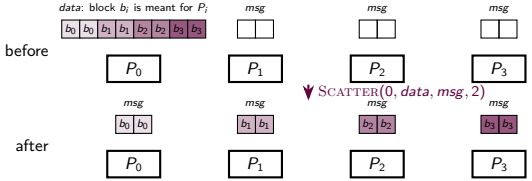


a) Design SCATTER$(k, data, msg, m)$ on the ring. Start with the naive version: the root keeps its block and sends all the others to its right neighbour, which keeps its block and sends the rest on, and so on. Then make the whole operation a pipeline: in which order should the root send the blocks, so that no block waits behind another one? Finally, write a third version made of $p-1$ calls to the SENDTO of Exercise 1.

b) Find the communication costs of the three versions. Explain why the cost of the pipelined version is the same as that of BCAST although $p-1$ different blocks are sent, and when the version based on SENDTO is as good.

Exercise 6: Scatter on a ring

In many applications, the data are initially held by a single processor (the one that read them from a file, say) and must be distributed in equal parts to all the processors, instead of starting from an already distributed configuration. SCATTER$(k, data, msg, m)$ does exactly this: the root P_k holds an array *data* of $p\,m$ elements, made of p consecutive blocks of m elements, the block $b_i = data[i\,m \ldots (i+1)m-1]$ being meant for P_i; at the end, every P_i holds its block b_i in its array *msg* of m elements.



a) Design SCATTER$(k, data, msg, m)$ on the ring. Start with the naive version: the root keeps its block and sends all the others to its right neighbour, which keeps its block and sends the rest on, and so on. Then make the whole operation a pipeline: in which order should the root send the blocks, so that no block waits behind another one? Finally, write a third version made of $p-1$ calls to the SENDTO of Exercise 1.

b) Find the communication costs of the three versions. Explain why the cost of the pipelined version is the same as that of BCAST although $p-1$ different blocks are sent, and when the version based on SENDTO is as good.

Exercise 6(a): the naive version, one link at a time

- The root keeps its block and sends the $p-1$ others, **in ring order**, to its right neighbour (lines 4–9; contiguous in *data* when $k=0$, copied into *tmp* otherwise).
- The processor at distance d receives $p-d$ blocks, its own first (line 10), keeps it (line 11) and **forwards the $p-d-1$ others** (line 12); P_{k-1} receives its block alone.
- Simple, but **a single link busy at a time**: each processor waits for the whole message before sending the rest on, and the blocks are transmitted again on every link they cross.

SCATTER(k, *data*, *msg*, m), NAIVE

```
1:  q ← MyRank(); p ← NumProcs()
2:  d ← (q − k) mod p                          ▶ distance to the root
3:  if q = k then
4:    msg[0 ... m − 1] ← data[k m ...]          ▶ its own block
5:    for i = 1, ..., p − 1 do
6:      j ← (k + i) mod p
7:      tmp[(i − 1) m ... i m − 1] ← data[j m ...]
8:    Send(tmp, (p − 1) m)                      ▶ the others, in ring order
9:  else
10:   Recv(tmp, (p − d) m)                     ▶ its own block first
11:   msg[0 ... m − 1] ← tmp[0 ... m − 1]
12:   if d < p − 1 then Send(tmp[m ...], (p − d − 1) m)
```

Exercise 6(a): the pipelined idea, farthest block first

- Like the pipelined transfer, with the blocks as the pieces, but with two differences.
- The root starts from the **other end** of its array: the farthest block first, in the **reverse order of the distances**: b_{k-1} ($p-1$ links away), then b_{k-2}, ..., b_{k+1} last: at step t, the block of $P_{(k-t) \bmod p}$. A block never waits behind another one: the block sent before it has already left.
- A processor does not wait for all the pieces, it needs only one: it forwards the blocks that are not for it, and keeps the last block it receives: its own.
- From its first use on, every link carries a block at every step, and all the blocks arrive at step $p-1$.

step	1	2	3	4
$P_0 \to P_1$	b_4	b_3	b_2	b_1
$P_1 \to P_2$		b_4	b_3	b_2
$P_2 \to P_3$			b_4	b_3
$P_3 \to P_4$				b_4

$p = 5$, root P_0: the block on each link at each step

Exercise 6(a): SCATTER

- The root sends one block per step, in the reverse order of the distances: at step t, the block of $P_{(k-t) \bmod p}$ (lines 3–4), and copies its own block (line 5).
- P_q, at distance $d = (q - k) \bmod p$, receives its first block at step d (line 7), forwards every block it receives except the last one (lines 8–10, $p - d - 1$ of them), and keeps the last one, its own (line 11).
- Two buffers, **swapped** before every step: the block forwarded is not overwritten by the one arriving.

SCATTER(k, *data*, *msg*, m)

```
1:  q ← MyRank(); p ← NumProcs()
2:  if q = k then
3:    for t = 1, ..., p − 1 do
4:      Send(data[((k − t) mod p) m ...], m)
5:    msg[0 ... m − 1] ← data[k m ... k m + m − 1]
6:  else
7:    Recv(tempR, m)
8:    for t = 1, ..., (k − 1 − q) mod p do
9:      tempS ↔ tempR
10:     Send(tempS, m) ‖ Recv(tempR, m)
11:   msg[0 ... m − 1] ← tempR[0 ... m − 1]
```

Exercise 6(a): a third version, $p-1$ calls to SendTo

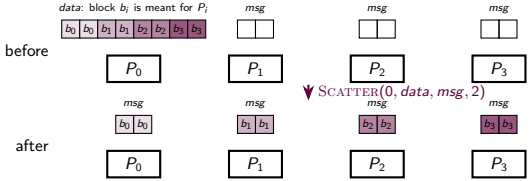
- The SendTo of Exercise 1 as a building block: $p-1$ transfers from the root **one after the other**, each one pipelined with its best r for its distance (line 4).
- Every transfer is a collective call of the whole ring; the blocking Recv of the intermediate processors make the transfers take place one after the other. Farthest first, so that an intermediate processor, whose *msg* every relayed block overwrites, ends with its own.
- The shortest to write; each transfer uses all the links of its path, but it is a **pipeline of its own**, started from scratch and drained before the next one starts.

Scatter$(k, data, msg, m)$, with SendTo

```
1: q ← MyRank(); p ← NumProcs()
2: for t = 1, ..., p − 1 do
3:    j ← (k − t) mod p; d ← p − t          ▶ farthest first
4:    r ← √((d − 1) m b/L                   ▶ the r* of 1(c)
5:    if q = k then msg[0 ... m − 1] ← data[j m ...]
6:    SendTo(k, j, msg, m, r)
7: if q = k then msg[0 ... m − 1] ← data[k m ...]
```

Exercise 6: Scatter on a ring

In many applications, the data are initially held by a single processor (the one that read them from a file, say) and must be distributed in equal parts to all the processors, instead of starting from an already distributed configuration. Scatter$(k, data, msg, m)$ does exactly this: the root P_k holds an array *data* of $p\,m$ elements, made of p consecutive blocks of m elements, the block $b_i = data[i\,m \ldots (i+1)m-1]$ being meant for P_i; at the end, every P_i holds its block b_i in its array *msg* of m elements.



a) Design Scatter$(k, data, msg, m)$ on the ring. Start with the naive version: the root keeps its block and sends all the others to its right neighbour, which keeps its block and sends the rest on, and so on. Then make the whole operation a pipeline: in which order should the root send the blocks, so that no block waits behind another one? Finally, write a third version made of $p-1$ calls to the SendTo of Exercise 1.

b) Find the communication costs of the three versions. Explain why the cost of the pipelined version is the same as that of Bcast although $p-1$ different blocks are sent, and when the version based on SendTo is as good.

Exercise 6(b): the naive version and the pipelined one

- Naive: step t is the message of $p-t$ blocks on the link $P_{k+t-1} \to P_{k+t}$, $L + (p-t)\,m\,b$, the steps one after the other:

$$T_{naive} = \sum_{t=1}^{p-1} \left(L + (p-t)\,m\,b\right) = (p-1)\,L + \frac{p\,(p-1)}{2}\,m\,b.$$

  The first message pushes the $p-1$ blocks through the root's link in $(p-1)\,m\,b$, but the same blocks are **transmitted again** on every link they cross: $p\,(p-1)/2$ block transmissions one after the other.

- Pipelined: b_{k-t} leaves the root at step t and crosses one link per step: link l at step $t+l-1$; it reaches P_{k-t} after $p-t$ links, at step $t+(p-t)-1 = p-1$: **all the blocks arrive at step $p-1$**, whatever their distance: $T_{comm} = (p-1)(L + m\,b)$, the same latency term, a transmission term $p/2$ times smaller.

- Same cost as the non-pipelined BCAST, although $p-1$ different blocks are sent: the $p-1$ blocks all leave the root through its **single link**, one per step, so $p-1$ steps for any algorithm. BCAST relays one block one link at a time; SCATTER keeps **all the links busy**: the reverse order makes it a pipeline, as in Exercise 2.

Exercise 6(b): the version with SendTo

- $p-1$ pipelined transfers one after the other, $c(d) = (d-1)\,L + m\,b + 2\sqrt{(d-1)\,L\,m\,b}$ each (1(c), distance d):

$$T_{\text{SendTo}} = \sum_{d=1}^{p-1} c(d) \approx (p-1)\,m\,b + \frac{(p-1)(p-2)}{2}\,L + 2\sqrt{L\,m\,b}\sum_{d=1}^{p-1}\sqrt{d-1}.$$

- The same transmission term $(p-1)\,m\,b$ as Scatter: once full, the pipeline of a SendTo **keeps the links as busy as Scatter**; the root's link, the bottleneck, carries every block once in both.
- In addition, the **start-up of $p-1$ pipelines** instead of one: $\approx p^2 L/2$ against $(p-1)\,L$, plus terms in $\sqrt{L\,m\,b}$ growing like $p^{3/2}$; negligible for large messages ($m\,b \gg p\,L$): **as good as the optimal version**. Small messages ($r^\star = 1$): $\frac{p\,(p-1)}{2}\,(L + m\,b)$, $p/2$ times more; the naive version is then as good as Scatter ($p-1$ messages on the critical path in both).

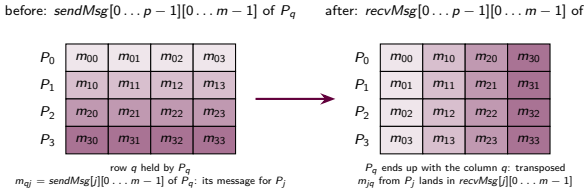
Algorithm	T_{comm}	T_{comp}	Remark
Scatter, naive	$(p-1)\,L + \frac{p\,(p-1)}{2}\,m\,b$	0	one link at a time
Scatter	$(p-1)(L + m\,b)$	0	one pipeline, = Bcast
Scatter with SendTo	$(p-1)\,m\,b + \frac{(p-1)(p-2)}{2}\,L + \ldots$	0	$p-1$ pipelines; optimal if $m\,b \gg p\,L$

Outline

1. Exercise 1: Pipelined point-to-point communication
2. Exercise 2: Pipelined broadcast on a ring
3. Exercise 3: Reduction on a ring
4. Exercise 4: Scan on a ring
5. Exercise 5: Matrix–vector product on a ring
6. Exercise 6: Scatter on a ring
7. Exercise 7: Transposition on a ring
8. Exercise 8: Grids, hypercubes and binary trees

Exercise 7: Transposition on a ring

Every processor P_q holds p messages of m elements in a $p \times m$ array $sendMsg[0 \dots p-1][0 \dots m-1]$: the row $sendMsg[j][0 \dots m-1]$ is the message meant for P_j ($sendMsg[q][0 \dots m-1]$ for itself). At the end, every P_q must hold in the row $recvMsg[j][0 \dots m-1]$ of a $p \times m$ array $recvMsg$ the message that P_j sent to it, for every j. Seen as a $p \times p$ array of messages whose row q is held by P_q, the operation transposes the array: P_q ends up with the column q. This exchange is the *all-to-all* of MPI.



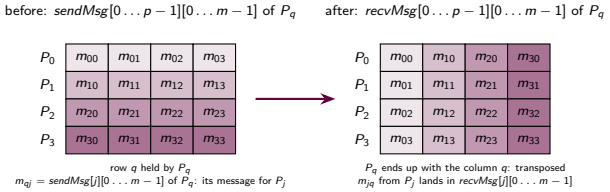
a) Design TRANSPOSITION($sendMsg$, $recvMsg$, m) on the ring, in two steps. First write SHIFT(src, dst, m, k): every P_q sends the m elements of its array src to $P_{(q+k) \bmod p}$ and receives in dst those of $P_{(q-k) \bmod p}$, a cyclic shift of the messages by k positions. Then use it: proceed in $p-1$ rounds; in round s, every processor sends the message meant for the processor at distance s.

b) Find its communication cost in terms of p, L, m and b.

c) Count the total number of link crossings that any algorithm must perform, and deduce a lower bound on the cost of a transposition on the ring. Is your algorithm optimal?

Exercise 7: Transposition on a ring

Every processor P_q holds p messages of m elements in a $p \times m$ array $sendMsg[0 \dots p-1][0 \dots m-1]$: the row $sendMsg[j][0 \dots m-1]$ is the message meant for P_j ($sendMsg[q][0 \dots m-1]$ for itself). At the end, every P_q must hold in the row $recvMsg[j][0 \dots m-1]$ of a $p \times m$ array $recvMsg$ the message that P_j sent to it, for every j. Seen as a $p \times p$ array of messages whose row q is held by P_q, the operation transposes the array: P_q ends up with the column q. This exchange is the *all-to-all* of MPI.



a) Design TRANSPOSITION($sendMsg$, $recvMsg$, m) on the ring, in two steps. First write SHIFT(src, dst, m, k): every P_q sends the m elements of its array src to $P_{(q+k) \bmod p}$ and receives in dst those of $P_{(q-k) \bmod p}$, a cyclic shift of the messages by k positions. Then use it: proceed in $p-1$ rounds; in round s, every processor sends the message meant for the processor at distance s.

b) Find its communication cost in terms of p, L, m and b.

c) Count the total number of link crossings that any algorithm must perform, and deduce a lower bound on the cost of a transposition on the ring. Is your algorithm optimal?

Exercise 7(a): SHIFT, k positions to the right

- The message of every processor travels k **hops**: at each hop, P_q sends the message it holds and receives the one of its left neighbour (line 3); the swap of line 4 makes the message just received the one to forward.
- After hop t, P_q holds the message of P_{q-t}; after k hops, the one of P_{q-k}, stored in *dst* (line 5). The copy of line 1 leaves *src* untouched.
- All the processors do the same thing at the same time: **every link carries one message at every hop**: SHIFT costs $k\,(L + m\,b)$.

SHIFT(src, dst, m, k)

1: $tempS[0 \dots m-1] \leftarrow src[0 \dots m-1]$
2: **for** $t = 1, \dots, k$ **do**
3: SEND$(tempS, m)$ $\|$ RECV$(tempR, m)$
4: $tempS \leftrightarrow tempR$
5: $dst[0 \dots m-1] \leftarrow tempS[0 \dots m-1]$

Exercise 7(a): TRANSPOSITION, $p-1$ shifts

- Round s: every processor sends the message for the processor s **links away**, the row $sendMsg[(q+s) \bmod p]$, with a SHIFT by s positions (lines 4–6).
- What it receives is the message of P_{q-s}, meant for $P_{q-s+s} = P_q$: the same call stores it in the row $recvMsg[(q-s) \bmod p]$. Line 2: the message for itself.
- A row of a 2-D array is passed by name like any array: $sendMsg[j]$ is the array $sendMsg[j][0], \ldots, sendMsg[j][m-1]$.

TRANSPOSITION($sendMsg$, $recvMsg$, m)

```
1: q ← MyRank(); p ← NumProcs()
2: recvMsg[q][0 ... m−1] ← sendMsg[q][0 ... m−1]
3: for s = 1, ..., p − 1 do
4:     j ← (q + s) mod p          ▶ the message for P_j
5:     i ← (q − s) mod p          ▶ the message from P_i
6:     Shift(sendMsg[j], recvMsg[i], m, s)
```

Exercise 7(a): the rounds, $p = 4$

université PARIS-SACLAY

- The message m_{qj} of P_q for P_j travels in round $(j - q) \bmod p$, over $(j - q) \bmod p$ links.
- Round 1: every processor sends to its right neighbour, one hop. Round 2: the messages for the processor two links away, two hops, forwarded once. Round 3: three hops.
- The diagonal of the array (the messages for oneself) needs no communication.

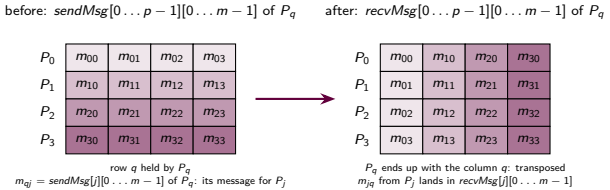
round of each message

	for P_0	for P_1	for P_2	for P_3
P_0	–	1	2	3
P_1	3	–	1	2
P_2	2	3	–	1
P_3	1	2	3	–

m_{qj} travels $(j - q) \bmod p$ links, in round $(j - q) \bmod p$

Exercise 7: Transposition on a ring

Every processor P_q holds p messages of m elements in a $p \times m$ array $sendMsg[0 \dots p-1][0 \dots m-1]$: the row $sendMsg[j][0 \dots m-1]$ is the message meant for P_j ($sendMsg[q][0 \dots m-1]$ for itself). At the end, every P_q must hold in the row $recvMsg[j][0 \dots m-1]$ of a $p \times m$ array $recvMsg$ the message that P_j sent to it, for every j. Seen as a $p \times p$ array of messages whose row q is held by P_q, the operation transposes the array: P_q ends up with the column q. This exchange is the *all-to-all* of MPI.



a) Design TRANSPOSITION($sendMsg$, $recvMsg$, m) on the ring, in two steps. First write SHIFT(src, dst, m, k): every P_q sends the m elements of its array src to $P_{(q+k) \bmod p}$ and receives in dst those of $P_{(q-k) \bmod p}$, a cyclic shift of the messages by k positions. Then use it: proceed in $p-1$ rounds; in round s, every processor sends the message meant for the processor at distance s.

b) Find its communication cost in terms of p, L, m and b.

c) Count the total number of link crossings that any algorithm must perform, and deduce a lower bound on the cost of a transposition on the ring. Is your algorithm optimal?

Exercise 7(b)–(c): cost and optimality

- Round s is a SHIFT by s positions: s hops of one message of m elements on every link, $s\,(L + m\,b)$: $T_{comm} = \sum_{s=1}^{p-1} s\,(L + m\,b) = \frac{p\,(p-1)}{2}\,(L + m\,b)$.
- Lower bound: the message of P_q for P_{q+s} must cross at least s links, whatever the algorithm; every processor has one message for each distance $s = 1, \ldots, p-1$, so its messages need $\sum_{s=1}^{p-1} s = p\,(p-1)/2$ link crossings, and all the processors together $p^2(p-1)/2$.
- The ring has p links and a link carries at most one message per $L + m\,b$: at least $\frac{p^2(p-1)/2}{p} = \frac{p\,(p-1)}{2}$ steps, hence $T_{comm} \geq \frac{p\,(p-1)}{2}\,(L + m\,b)$ for any algorithm that sends the messages whole.
- The algorithm reaches the bound: **optimal**, every link is busy at every step.

Algorithm	T_{comm}	T_{comp}	Remark
TRANSPOSITION	$p\,(p-1)(L + m\,b)/2$	0	optimal: every link busy at every step

Outline

1. Exercise 1: Pipelined point-to-point communication
2. Exercise 2: Pipelined broadcast on a ring
3. Exercise 3: Reduction on a ring
4. Exercise 4: Scan on a ring
5. Exercise 5: Matrix–vector product on a ring
6. Exercise 6: Scatter on a ring
7. Exercise 7: Transposition on a ring
8. Exercise 8: Grids, hypercubes and binary trees

Exercise 8: Grids, hypercubes and binary trees

Back to the parameters of the lecture, on three other topologies.

a) What is the diameter of a complete binary tree of height h (the root at depth 0, the leaves at depth h)?

b) On an $n \times n$ grid, how many different shortest paths are there between the processors at positions (x_1, y_1) and (x_2, y_2)?

c) Prove that the bisection width of the d-cube ($p = 2^d$ processors) is 2^{d-1}: exhibit a cut of 2^{d-1} links that splits it into two halves, then show that no cut with fewer links does.

Exercise 8: Grids, hypercubes and binary trees

Back to the parameters of the lecture, on three other topologies.

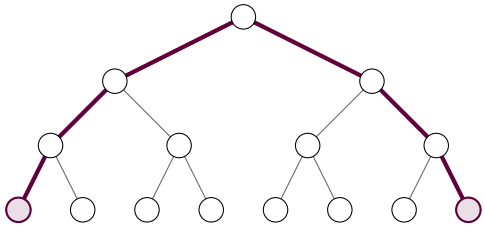
a) What is the diameter of a complete binary tree of height h (the root at depth 0, the leaves at depth h)?

b) On an $n \times n$ grid, how many different shortest paths are there between the processors at positions (x_1, y_1) and (x_2, y_2)?

c) Prove that the bisection width of the d-cube ($p = 2^d$ processors) is 2^{d-1}: exhibit a cut of 2^{d-1} links that splits it into two halves, then show that no cut with fewer links does.

Exercise 8(a): the diameter of a complete binary tree

- Two nodes are joined by the path that goes up to their **lowest common ancestor** and down again.
- The longest such path joins two leaves whose only common ancestor is the root: h links up, h links down.
- The diameter is $2h$; with $p = 2^{h+1} - 1$ nodes, $2h \approx 2 \log_2 p$: as good as the hypercube, with degree 3 at most, but the root is a bottleneck: bisection width 1.



h links up to the root, h links down: $2h = 6$

height $h = 3$: the leaves are at depth 3

Exercise 8: Grids, hypercubes and binary trees

Back to the parameters of the lecture, on three other topologies.

a) What is the diameter of a complete binary tree of height h (the root at depth 0, the leaves at depth h)?

b) On an $n \times n$ grid, how many different shortest paths are there between the processors at positions (x_1, y_1) and (x_2, y_2)?

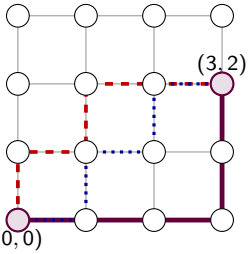
c) Prove that the bisection width of the d-cube ($p = 2^d$ processors) is 2^{d-1}: exhibit a cut of 2^{d-1} links that splits it into two halves, then show that no cut with fewer links does.

Exercise 8(b): shortest paths on a grid

- $\Delta x = |x_2 - x_1|$, $\Delta y = |y_2 - y_1|$. A shortest path never moves away from its destination: exactly Δx horizontal steps and Δy vertical steps, $\Delta x + \Delta y$ in total, **in any order**; every order stays inside the grid.
- A path is determined by the choice of which of its $\Delta x + \Delta y$ steps are horizontal: the number of shortest paths is

$$\binom{\Delta x + \Delta y}{\Delta x} = \frac{(\Delta x + \Delta y)!}{\Delta x!\, \Delta y!}.$$

- Many shortest paths: the grid can spread the traffic between two nodes, unlike the ring, where there is a single path.



three of the $\binom{5}{2} = 10$ paths:
RRRUU, URURR, RURUR

from $(0, 0)$ to $(3, 2)$: $\Delta x = 3$, $\Delta y = 2$, 5 steps, 2 of them vertical

Exercise 8: Grids, hypercubes and binary trees

Back to the parameters of the lecture, on three other topologies.

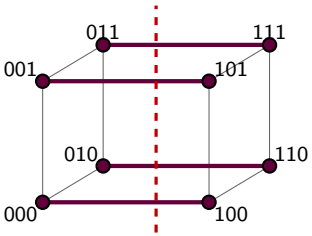
a) What is the diameter of a complete binary tree of height h (the root at depth 0, the leaves at depth h)?

b) On an $n \times n$ grid, how many different shortest paths are there between the processors at positions (x_1, y_1) and (x_2, y_2)?

c) Prove that the bisection width of the d-cube ($p = 2^d$ processors) is 2^{d-1}: exhibit a cut of 2^{d-1} links that splits it into two halves, then show that no cut with fewer links does.

Exercise 8(c): a cut of 2^{d-1} links

- Split the nodes by their **top bit**: Q_0 holds the 2^{d-1} nodes $0b_{d-2}\dots b_0$, Q_1 the 2^{d-1} nodes $1b_{d-2}\dots b_0$; each is a $(d-1)$-cube.
- The only links between them are the 2^{d-1} links of the top dimension, between each node of Q_0 and its twin in Q_1.
- Removing them splits the d-cube into two halves: the bisection width is **at most** 2^{d-1}.
- It remains to show that no cut with fewer links splits the cube into two halves.



Q_0: top bit 0 Q_1: top bit 1

the $2^{d-1} = 4$ links of the top dimension (thick) separate Q_0 and Q_1

Exercise 8(c): no cut does better

Lemma: in a d-cube, every set S of $s \leq 2^{d-1}$ nodes has at least s links *leaving* it (links with exactly one end in S). A half of a bisection is a set of 2^{d-1} nodes and the links leaving it are the links of the cut: at least 2^{d-1} of them, so $L_B = 2^{d-1}$.

- **Induction on d.** $d = 1$: S is one of the two nodes, the single link leaves it.
- $d > 1$: split into Q_0 and Q_1 by the top bit, $S_0 = S \cap Q_0$, $S_1 = S \cap Q_1$, $s_0 + s_1 = s$. The links leaving S: those leaving S_0 inside Q_0, those leaving S_1 inside Q_1, and the links of the top dimension with one end in S and the other outside: **at least** $|s_0 - s_1|$ of them (if every node of S_0 had its twin in S_1, then $s_1 \geq s_0$; symmetrically).
- If $s_0, s_1 \leq 2^{d-2}$: induction in the two $(d-1)$-cubes gives $\geq s_0 + s_1 + |s_0 - s_1| \geq s$ links.
- Otherwise, say $s_0 > 2^{d-2}$, hence $s_1 < 2^{d-2}$: the links leaving S_0 inside Q_0 are also those leaving its **complement** $Q_0 \setminus S_0$, of size $2^{d-1} - s_0 < 2^{d-2}$: at least $2^{d-1} - s_0$ by induction; plus $\geq s_1$ for S_1 and $\geq s_0 - s_1$ top-dimension links: $\geq 2^{d-1} - s_0 + s_1 + s_0 - s_1 = 2^{d-1} \geq s$.
- Hence the bisection width of the d-cube is exactly $2^{d-1} = p/2$, as in the table of the lecture: the cut of one dimension is optimal.

Contact

Oguz Kaya
Université Paris-Saclay and LISN, Paris, France
oguz.kaya@universite-paris-saclay.fr
www.oguzkaya.com